INTELLIGENT COGNITIVE ALARM PLATFORM

G.SAI SOUMYA SRI

Table of Contents

1.Project Description

The Intelligent Cognitive Alarm Platform is an AI-powered alarm app that requires users to solve a puzzle — math, logic, memory, or riddle — before the alarm can be dismissed. The primary goal of this project is to help users build consistent wake-up habits and reduce oversleeping by adapting challenge difficulty based on their behavior.

This project implements a full-stack architecture, combining a FastAPI backend, a PostgreSQL database, and an AI/ML engine that adjusts challenge difficulty based on snooze patterns, sleep schedule, and past performance

2.Environment Setup

The database layer was developed and deployed using Supabase, a hosted PostgreSQL platform, accessed through its built-in SQL Editor and Table Editor. The implementation was done locally using Visual Studio Code as the primary IDE.
The key tools and configurations used include:`

Supabase (PostgreSQL): The core hosted database platform used to create, manage, and query all tables in the schema.

SQL Editor (Supabase): Used to write and execute CREATE TABLE scripts, ENUM type definitions, and foreign key constraints directly against the hosted database.

pgcrypto extension: Enabled to auto-generate UUID primary keys (gen_random_uuid()) for every table.

Table Editor (Supabase): Used to visually verify that each table, column, and relationship was created correctly after running the scripts.

VS Code: Used as the primary code editor for writing and organizing SQL scripts before running them in Supabase.

3.Proposed system / Architecture

The Intelligent Cognitive Alarm Platform follows a layered architecture that separates the user-facing app from the backend logic and the database. The system operates as follows:

Client Layer: Users interact with the app through a mobile app (React Native) or web dashboard (React.js) — setting alarms, solving challenges, and viewing habit/progress reports.

API Gateway: All requests pass through a FastAPI gateway that handles routing, authentication, rate limiting, and request validation before reaching the backend services.

Backend / Microservices: Python and FastAPI power individual services for users, alarms, challenges, adaptive difficulty, verification, and analytics — secured using JWT and OAuth2.

AI/ML Engine: Machine learning models (Scikit-learn, XGBoost, TensorFlow, PyTorch) generate personalized challenges and adjust difficulty based on user behavior, sleep patterns, and snooze history.

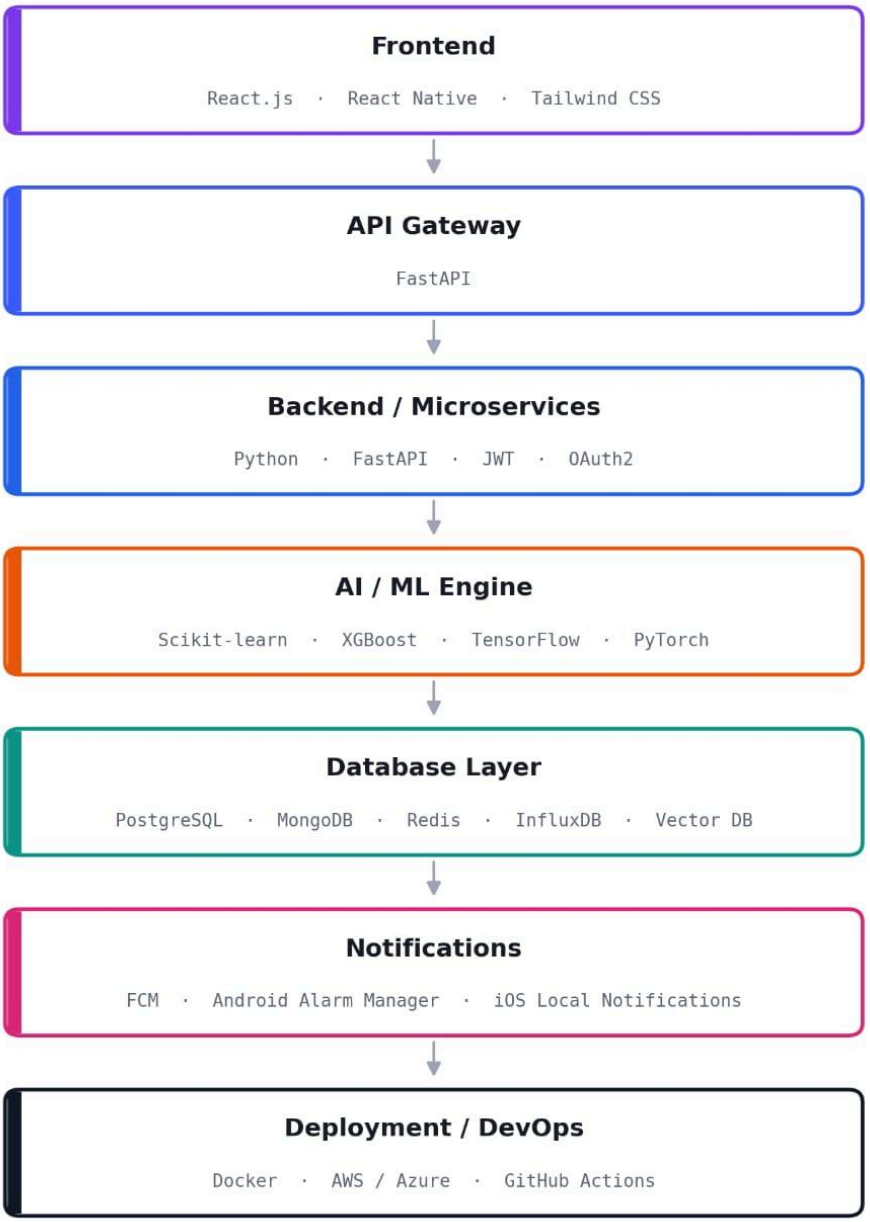
Database Layer (my contribution): PostgreSQL, hosted on Supabase, stores all persistent data — users, alarms, challenges, habit scores, notifications, and reports — and is the single source of truth every other layer reads from and writes to.

Notification Layer: Firebase Cloud Messaging and native alarm managers (Android/iOS) deliver reminders and alerts.

Deployment: The system is containerized with Docker and deployed to cloud platforms (AWS/Azure).

INTELLIGENT COGNITIVE ALARM

PLATFORM — SCHEMA



4.Database Schema Design

The database is designed as a relational PostgreSQL schema, hosted on Supabase, with a total of 15 tables organized into 3 logical phases matching the project's weekly build plan.

Phase 1 — Core Foundation (6 tables): users, password_reset_tokens, alarms, alarm_triggers, challenges, notifications

Phase 2 — Intelligence & Analytics (5 tables): challenge_attempts, sleep_logs, habit_scores, recommendations, goal_metrics

Phase 3 — Reports, Admin & Platform (4 tables): reports, audit_logs, coach_assignments, platform_settings

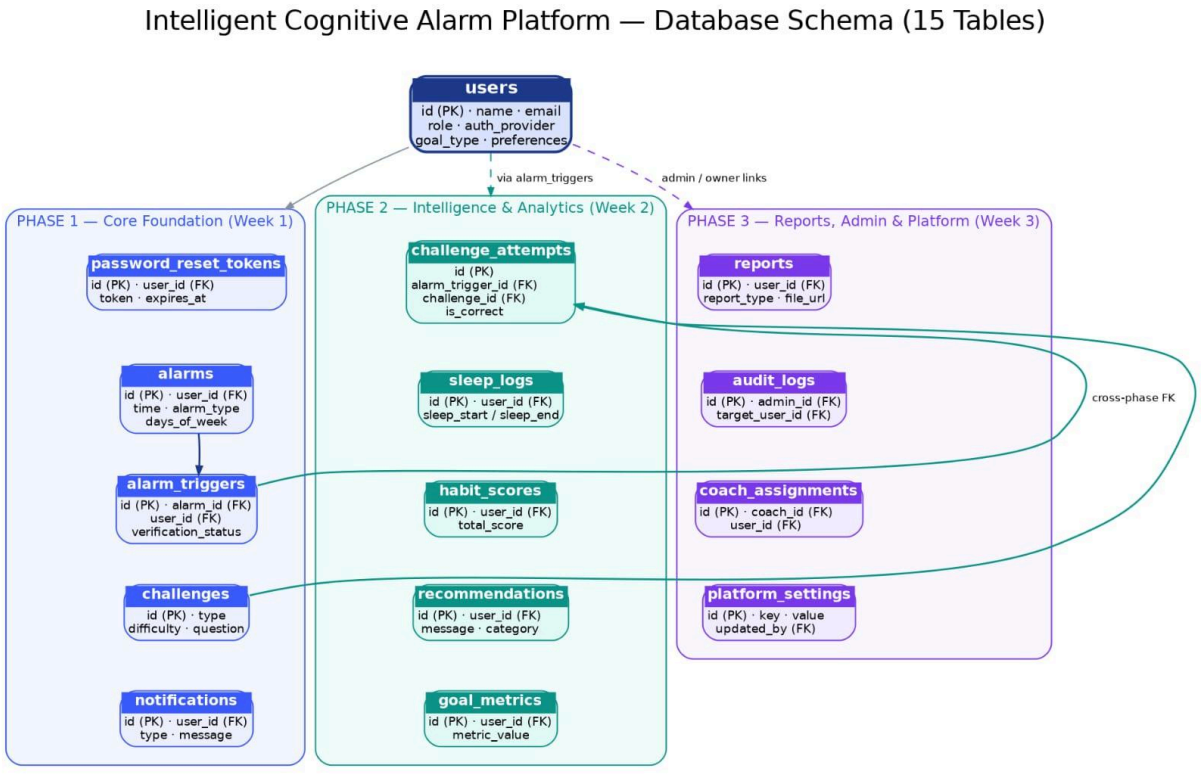
The users table is the central identity table and parent of nearly every other table in the schema — it stores authentication details (email/Google login), user role (user, wellness coach, admin), and personalization preferences (wake time, sleep duration, difficulty preference).

The core relationship chain that drives the entire alarm-and-challenge flow is:

users → alarms → alarm_triggers → challenge_attempts → challenges

Every alarm belongs to a user, every trigger (firing event) belongs to an alarm, and every challenge attempt is logged against both the trigger and the challenge it was answering. This structure allows the platform to track snooze behavior, verify wake-up completion, and later calculate habit scores from the same data.

UUID primary keys are used throughout for safe, collision-free record generation, and ENUM types (e.g., role, alarm_type, difficulty, verification_status) enforce valid values directly at the database level.



5. Work completed

Out of the 15 planned tables, 6 tables (Phase 1 — Core Foundation) have been designed and deployed on Supabase for Milestone 1: users, password_reset_tokens, alarms, alarm_triggers, challenges, and notifications.

Each table was created using PostgreSQL CREATE TABLE scripts with UUID primary keys, VARCHAR types for fixed-choice fields, and foreign key constraints linking back to users and related tables. All 6 scripts were run successfully in the Supabase SQL Editor and verified in the Table Editor.

```sql
CREATE TABLE notifications (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  type VARCHAR(60) NOT NULL,
  message TEXT NOT NULL,
  sent_at TIMESTAMP DEFAULT NOW(),
  read_at TIMESTAMP
);
```

```sql
CREATE TABLE users (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash VARCHAR(255),
  auth_provider VARCHAR(20) DEFAULT 'email',
  google_id VARCHAR(255) UNIQUE,
  role VARCHAR(20) DEFAULT 'user',
  goal_type VARCHAR(20),
  preferred_wake_time TIME,
  sleep_duration FLOAT,
  timezone VARCHAR(50) DEFAULT 'Asia/Kolkata',
  difficulty_preference VARCHAR(20) DEFAULT 'beginner',
  is_active BOOLEAN DEFAULT true,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW());
```

```sql
CREATE TABLE alarms (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  time TIME NOT NULL,
  alarm_type VARCHAR(20) DEFAULT 'daily',
  days_of_week INTEGER[],
  is_active BOOLEAN DEFAULT true,
  label VARCHAR(100),
  created_at TIMESTAMP DEFAULT NOW()
);
```

```sql
CREATE TABLE challenges (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  type VARCHAR(30) NOT NULL,
  goal_type VARCHAR(20),
  difficulty VARCHAR(20) DEFAULT 'beginner',
  question TEXT NOT NULL,
  correct_answer TEXT NOT NULL,
  source VARCHAR(20) DEFAULT 'question_bank',
  embedding_ref VARCHAR(120),
  created_at TIMESTAMP DEFAULT NOW()
);
```

```sql
CREATE TABLE password_reset_tokens (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token VARCHAR(255) UNIQUE NOT NULL,
  expires_at TIMESTAMP NOT NULL,
  used BOOLEAN DEFAULT false,
  created_at TIMESTAMP DEFAULT NOW()
);
```

```sql
CREATE TABLE alarm_triggers (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  alarm_id UUID NOT NULL REFERENCES alarms(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  triggered_at TIMESTAMP DEFAULT NOW(),
  dismissed_at TIMESTAMP,
  verification_status VARCHAR(20) DEFAULT 'pending',
  snooze_count INTEGER DEFAULT 0,
  total_attempts INTEGER DEFAULT 0
);
```

6 .Conclusion and Next steps

successfully establishes the database foundation for the Intelligent Cognitive Alarm Platform. The users table is live as the central identity source, and the core alarm-and-challenge relationship chain — alarms → alarm_triggers → challenges — is fully deployed on Supabase with proper foreign key constraints and ENUM-based validation. Six of the fifteen planned tables are complete, giving the platform a working foundation for authentication, alarm scheduling, and challenge delivery.

Next Steps :

1. Implement user authentication (JWT + OAuth2) and role-based access control.
2. Build out alarm scheduling logic, including recurring and smart-adaptive alarms.
3. Apply Row-Level Security (RLS) policies on all tables to enforce per-user data access.
4. Design and deploy Phase 2 tables: challenge_attempts, sleep_logs, habit_scores, recommendations, goal_metrics.
5. Begin connecting the habit scoring formula to real data once sleep_logs and challenge_attempts are live